

SQL 注入漏洞报告 (Lab3: UNION 攻击确定列数)

1. 漏洞名称

SQL Injection (SQL 注入) — UNION 联合查询确定列数

2. 漏洞等级

高危 (High)

3. 漏洞描述

该网站商品分类查询功能存在 SQL 注入漏洞。攻击者可以通过 UNION 关键字将恶意查询附加到原始查询上。由于 UNION 要求前后查询的列数必须相同，攻击者可以通过输入不同数量的 NULL 来探测原始查询包含多少列，为后续跨表窃取数据做准备。

4. 漏洞位置

请求方式: GET /filter?category=...

漏洞参数: category

5. 漏洞复现过程

第一步：抓包与列数探测（注入测试）

将 category 参数修改为构造的 UNION Payload，逐步增加 NULL 的数量进行探测。

- 试错探测 Payload: category=Tech+gifts'+UNION+SELECT+null,null,null--
- 列数匹配 Payload: category=Tech+gifts'+UNION+SELECT+null,null,null,null--



截图 1：列数不匹配导致报错（500 服务器错误）

说明：当 UNION 查询的列数（比如 3 列）与原始查询列数（比如 4 列）不一致时，数据库会抛出异常，导致页面 500 报错。这证明注入点存在且 SQL 语句已被执行。



截图 2：列数匹配成功（页面正常显示）

说明：当 UNION 查询使用 4 个 NULL 时 (SELECT null,null,null,null)，列数匹配成功，页面正常渲染。这确认了原始查询包含 4 列。



截图 3：实验完成确认

说明：靶场服务器确认漏洞攻击验证成功。

第二步：漏洞原理

原 SQL:

sql

```
SELECT * FROM products WHERE category='Tech+gifts'
```

攻击者输入 Payload 后，拼接的 SQL 变为了：

sql

```
SELECT * FROM products WHERE category='Tech+gifts' UNION SELECT  
null,null,null,null -- '
```

- UNION **关键字**用于合并两个 SELECT 语句的结果。
- NULL 用作占位符，因为攻击者不知道原表列的数据类型，NULL 兼容所有数据类型。
- 只有当前后两个 SELECT 语句的**列数完全相等**（本题是 4 列），页面才会正常渲染（图 1 的效果）；若不相等（如 3 列），数据库会报错（图 2 500 错误）。

6. 漏洞影响

虽然这个 Lab 本身只是“探测列数”，但它是整个 SQL 注入链的关键一步：

- 一旦确定了列数，攻击者可以**立刻替换 NULL 为具体的列名**，执行后续的跨表检索攻击。

- 可能进一步导致数据库中的用户名、密码等敏感信息泄露。

7. 修复建议

- ① **使用参数化查询（预编译）**：禁止将用户输入直接拼接到 SQL 语句中。
- ② **最小权限原则**：数据库连接账号仅具备必要查询权限，禁用 UNION 等危险操作权限（如 MySQL 中可禁用 SELECT 权限，或使用 Web 应用防火墙拦截）。
- ③ **统一错误处理**：当发生 SQL 错误时，前端只显示“系统繁忙”或“内部错误”，**绝对不要**将原始的数据库报错信息（如列数不匹配的误差）返回给前端。

8. 工具

- Google Chrome / Firefox 浏览器（直接修改 URL 参数）
- PortSwigger Web Security Academy（靶场平台）